

MOMO Quant

Part 5 — AI & Machine Learning Design Document

Version: 1.0 Draft

AI Service: Python

Backend Integration: .NET 8 Web API

Primary Goal: Use AI to improve decision quality, not to blindly control trades.

1. AI Design Principle

MOMO Quant must not be built as a black-box AI trading bot.

That would be dangerous and hard to debug.

The correct design is:

```
Deterministic Strategy Logic
      +
AI Assistance
      +
Strict Risk Management
      =
Controlled Trading System
```

AI will support the platform by:

- classifying market conditions
- scoring trade confidence
- optimizing strategy parameters
- detecting abnormal market conditions
- explaining decisions
- analyzing historical performance

AI must not directly place orders.

AI must not bypass the risk engine.

AI must not rewrite strategy rules automatically.

2. AI Responsibilities

The AI layer will be responsible for five main areas:

1. Market Regime Detection
2. Trade Confidence Scoring
3. Parameter Optimization
4. Anomaly Detection
5. Trade Explanation

Each responsibility must be implemented as a separate module.

3. AI Service Architecture

The Python AI service will run separately from the .NET backend.

Recommended structure:

```
momo-ai/  
  app/  
    main.py  
    api/  
      regime_routes.py  
      confidence_routes.py  
      optimization_routes.py  
      anomaly_routes.py  
      explanation_routes.py  
  
    core/  
      config.py  
      logging.py  
      schemas.py  
  
    services/  
      regime_service.py  
      confidence_service.py  
      optimization_service.py  
      anomaly_service.py  
      explanation_service.py  
  
    models/  
      regime_model.py
```

```
confidence_model.py
optimization_model.py

features/
  feature_builder.py
  candle_features.py
  indicator_features.py
  trade_features.py

storage/
  model_store.py

tests/
```

The first version can use rule-based scoring and statistical logic before adding complex machine learning.

Do not start with deep learning.

4. AI Integration Flow

The .NET backend sends structured market context to the Python AI service.

Flow:

```
Market Data
|
v
Indicator Engine
|
v
Strategy Engine
|
v
.NET Backend builds AI request
|
v
Python AI Service
|
v
AI response saved to database
|
v
Risk Engine
|
```

v
Execution Engine

The AI result must always be stored in `AiDecisions`.

5. Market Regime Detection

5.1 Purpose

Market regime detection determines what type of market is currently active.

This is important because different strategies work in different conditions.

For example:

Trending market	-> EMA Pullback
Ranging market	-> VWAP Mean Reversion
Sweep/reversal market	-> Liquidity Sweep
Choppy market	-> No trade or reduced confidence

5.2 Supported Market Regimes

The system must classify markets into:

- Trending
- Ranging
- Breakout
- Reversal
- HighVolatility
- LowVolatility
- Choppy
- Abnormal
- Unknown

5.3 Input Data

Market regime detection should receive:

```
{
  "symbol": "BTCUSDT",
  "timeframe": "3m",
  "higherTimeframe": "15m",
  "candles": [],
  "indicators": {
    "ema20": 0,
    "ema50": 0,
    "ema200": 0,
    "vwap": 0,
    "rsi14": 0,
    "atr14": 0,
    "volumeSma20": 0
  },
  "marketStructure": {
    "higherHighs": 0,
    "higherLows": 0,
    "lowerHighs": 0,
    "lowerLows": 0
  }
}
```

5.4 Initial Rule-Based Logic

MVP should begin with explainable rules.

Example:

```
If EMA20 > EMA50 > EMA200
and price is above EMA50
and ATR is normal
and structure shows higher highs / higher lows
then regime = Trending
```

Example:

```
If price repeatedly crosses VWAP
and ATR is low/moderate
and candles overlap heavily
then regime = Ranging
```

Example:

If ATR suddenly expands
and volume is above average
and candle body size is unusually large
then regime = HighVolatility or Breakout

5.5 Output

```
{  
  "marketRegime": "Trending",  
  "confidence": 82,  
  "explanation": "EMA alignment is bullish, price is above EMA50, and market  
structure shows higher highs and higher lows."  
}
```

6. Trade Confidence Scoring

6.1 Purpose

The confidence engine converts strategy signals and market context into a final confidence score.

This avoids the bad design of requiring all strategies to agree.

Instead, the bot uses weighted scoring.

6.2 Input

```
{  
  "symbol": "BTCUSDT",  
  "timeframe": "3m",  
  "marketRegime": "Trending",  
  "strategySignals": [  
    {  
      "strategyCode": "EMA_PULLBACK",  
      "direction": "Long",  
      "strength": 80,  
      "reason": "Price pulled back to EMA20 and bullish candle closed."  
    }  
  ],  
}
```

```
"indicators": {},  
"riskContext": {  
  "dailyPnl": 0,  
  "consecutiveLosses": 0,  
  "openPositions": 0,  
  "spreadPercent": 0.01  
}  
}
```

6.3 Initial Confidence Scoring Model

MVP should use a transparent weighted model.

Example scoring:

Strategy signal strength	0-40 points
Market regime alignment	0-20 points
Higher timeframe confirmation	0-15 points
Volume confirmation	0-10 points
Volatility quality	0-10 points
Spread/execution quality	0-5 points

Maximum score:

100

6.4 Trade Approval Thresholds

Recommended default thresholds:

```
Confidence >= 80 -> Trade allowed  
Confidence 70-79 -> Watchlist / no trade  
Confidence < 70 -> Reject
```

These values must be configurable.

6.5 Output

```
{
  "confidenceScore": 86,
  "tradeAllowed": true,
  "preferredStrategyCode": "EMA_PULLBACK",
  "riskAdjustment": "Normal",
  "explanation": "EMA pullback signal aligns with trending regime, higher
timeframe confirms direction, and volatility is acceptable."
}
```

7. Strategy Weighting

Each strategy must have regime-specific weighting.

Example:

```
Trending Market:
  EMA_PULLBACK          45%
  LIQUIDITY_SWEEP       25%
  VWAP_MEAN_REVERSION   10%

Ranging Market:
  VWAP_MEAN_REVERSION   45%
  LIQUIDITY_SWEEP       30%
  EMA_PULLBACK          10%

Choppy Market:
  All strategies reduced or disabled
```

This must be configurable.

Do not hardcode final weights permanently.

8. Parameter Optimization

8.1 Purpose

Parameter optimization helps find better settings for each strategy.

Examples:

```
EMA periods
ATR multiplier
Minimum confidence threshold
VWAP deviation threshold
Liquidity sweep size
Stop loss multiplier
Take profit multiplier
Order timeout seconds
```

8.2 Optimization Must Not Overfit

This is a major risk.

Bad optimization:

```
Find the best settings for January and use them blindly.
```

Correct optimization:

```
Train on one period.
Validate on another period.
Test on unseen data.
Only accept parameters that perform well across multiple periods.
```

8.3 Required Validation Method

Use walk-forward validation.

Example:

```
Train: January
Validate: February
Test: March

Then roll forward:

Train: February
```

Validate: March
Test: April

The system should reject parameter sets that only work in one isolated period.

8.4 Optimization Metrics

Do not optimize only for net profit.

Use multiple metrics:

Net PnL
Profit factor
Max drawdown
Win rate
Expectancy
Average R multiple
Trade count
Sharpe ratio
Sortino ratio
Fee impact
Missed order rate

A strategy that makes high profit with huge drawdown should not be trusted.

8.5 Output

```
{
  "strategyCode": "EMA_PULLBACK",
  "symbol": "BTCUSDT",
  "timeframe": "3m",
  "recommendedParameters": {
    "EmaFastPeriod": 20,
    "EmaSlowPeriod": 50,
    "AtrMultiplier": 1.6,
    "MinConfidence": 82
  },
  "validationMetrics": {
    "netPnl": 1200,
    "profitFactor": 1.45,
    "maxDrawdown": 6.2,
    "tradeCount": 140
  }
}
```

```
},  
  "accepted": true,  
  "reason": "Parameter set performed consistently across validation windows."  
}
```

9. Anomaly Detection

9.1 Purpose

Anomaly detection protects the bot from abnormal conditions.

Examples:

```
Exchange latency spike  
WebSocket data gap  
Massive spread expansion  
Abnormal candle size  
Sudden volume shock  
API failures  
Extreme volatility  
Possible news-driven move
```

When anomalies are detected, the bot should reduce risk or stop taking new trades.

9.2 Severity Levels

```
Low  
Medium  
High  
Critical
```

Recommended action:

```
Low      -> Continue  
Medium   -> Reduce confidence  
High     -> Block new trades  
Critical -> Emergency stop
```

9.3 Output

```
{
  "isAnomaly": true,
  "severity": "High",
  "reason": "Spread is 5x above normal and ATR has expanded sharply.",
  "recommendedAction": "BlockNewTrades"
}
```

10. Trade Explanation

10.1 Purpose

Every trade must be explainable.

This is not optional.

Without explanation, debugging becomes guesswork.

10.2 Explanation Input

```
{
  "signal": {},
  "aiDecision": {},
  "riskDecision": {},
  "executionDecision": {}
}
```

10.3 Explanation Output

```
{
  "explanation": "Long trade was approved because the market regime was  
Trending, EMA Pullback produced a strong bullish signal, confidence was 86,  
higher timeframe confirmed direction, and risk limits were valid."
}
```

10.4 No-Trade Explanation

The system must also explain rejected trades.

Example:

```
{  
  "explanation": "No trade was taken because confidence was 68, below the  
  required threshold of 80. The market was choppy and spread was above the  
  configured limit."  
}
```

Rejected setups are as important as executed trades.

11. Feature Engineering

The AI service should build features from raw candles and indicators.

Feature categories:

```
Price features  
Volume features  
Volatility features  
Trend features  
Market structure features  
Strategy signal features  
Risk context features  
Execution quality features  
Historical performance features
```

11.1 Example Features

```
price_above_ema20  
price_above_ema50  
ema20_slope  
ema50_slope  
atr_percent  
volume_vs_sma  
distance_from_vwap  
rsi_value
```

```
candle_body_percent
upper_wick_percent
lower_wick_percent
recent_swing_high_distance
recent_swing_low_distance
spread_percent
consecutive_losses
daily_pnl_percent
```

12. Data Storage for AI

The following data must be stored for AI learning:

```
Candles
Indicators
Strategy signals
AI decisions
Risk decisions
Orders
Order fills
Trades
Missed orders
Rejected trades
Market regime labels
Performance reports
```

Do not train only on executed trades.

That creates biased learning.

The model must also learn from rejected setups and missed orders.

13. Model Versioning

Every AI model version must be tracked.

Store:

```
Model name
Model version
```

Training date
Training data range
Feature set version
Metrics
Active status
Notes

A future model must not silently replace the current model.

Every model change should be auditable.

14. Human Approval for Model Updates

In MVP, AI parameter recommendations must require manual approval.

Flow:

```
AI recommends parameters
  |
  v
Dashboard shows comparison
  |
  v
Developer reviews
  |
  v
Developer approves or rejects
  |
  v
System applies changes
```

Do not allow automatic model-driven strategy changes in early versions.

That is how systems become unstable.

15. AI Safety Rules

The AI system must follow these rules:

1. AI cannot directly place orders.
2. AI cannot bypass the risk engine.

3. AI cannot enable live trading.
 4. AI cannot change risk limits automatically.
 5. AI cannot deploy new parameters without approval in MVP.
 6. AI output must be logged.
 7. AI confidence must be explainable.
 8. AI failures must not crash the trading engine.
 9. If AI service is unavailable, the bot should either:
 10. fall back to rule-based scoring, or
 11. stop trading, depending on configuration.
 12. AI must not be trusted blindly.
-

16. AI Failure Handling

The backend must handle:

```
AI service timeout
AI service unavailable
Invalid AI response
Low AI confidence
AI response conflict
Model version missing
Feature generation failure
```

Recommended default behavior:

```
Backtesting -> allow fallback mode
Replay      -> allow fallback mode
Paper       -> stop new trades or use configured fallback
Live        -> block new trades
```

For live trading, AI failure should be treated conservatively.

17. AI Performance Evaluation

AI must be evaluated continuously.

Reports should show:

- Average confidence score
- Win rate by confidence bucket
- PnL by confidence bucket
- Confidence vs outcome accuracy
- Regime classification performance
- Best regime
- Worst regime
- False approvals
- False rejections
- AI-approved losses
- AI-rejected winners

Example confidence buckets:

- 70-79
- 80-89
- 90-100

This tells whether AI confidence actually means anything.

18. AI Dashboard Requirements

The dashboard must include AI pages:

- AI Decisions
- AI Confidence Analysis
- Market Regime Analysis
- Parameter Optimization
- Model Versions
- AI Errors
- AI Recommendations

Each AI decision page should show:

- Symbol
- Timeframe
- Strategy
- Market regime
- Confidence
- Trade allowed
- Risk adjustment

Explanation
Raw request
Raw response
Trade outcome if available

19. MVP AI Scope

The first AI version should include:

Rule-based market regime detection
Weighted confidence scoring
Simple anomaly detection
Trade explanation
AI decision logging
AI dashboard views
Manual parameter recommendation review

Do not start MVP with:

Deep learning
Reinforcement learning
Autonomous model deployment
Autonomous risk adjustment
Autonomous live trading

That would be unnecessary complexity and dangerous.

20. Later AI Enhancements

Future versions may add:

Supervised learning models
Gradient boosting models
Walk-forward optimization engine
Automated feature importance analysis
Market regime clustering
News sentiment filter
Cross-symbol correlation model
Volatility forecasting

```
Fill probability model for maker orders  
Adaptive strategy selection
```

These should only be added after the MVP produces reliable data.

21. Recommended Python Libraries

Initial Python stack:

```
FastAPI  
Pydantic  
pandas  
numpy  
scikit-learn  
joblib  
uvicorn  
pytest
```

Optional later:

```
xgboost  
lightgbm  
optuna  
statsmodels  
mlflow
```

Do not add too many libraries at the start.

The first version must be understandable.

22. AI Development Order

Build AI in this order:

1. Python FastAPI service skeleton
2. Health endpoint
3. Pydantic request/response models
4. Feature builder
5. Rule-based market regime detector

6. Weighted confidence scorer
7. Simple anomaly detector
8. Explanation generator
9. .NET AI service client
10. AI decision database logging
11. AI dashboard pages
12. Parameter recommendation module
13. Walk-forward optimizer later

23. AI Testing Requirements

AI service must have tests for:

Market regime classification
Confidence scoring
Anomaly detection
Invalid input handling
Missing candle handling
Extreme value handling
Response schema validation
Fallback behavior

Example tests:

Trending candle set returns Trending
Choppy candle set returns Choppy
High spread returns anomaly
Weak signal returns low confidence
Strong aligned signal returns high confidence
Invalid candle data returns validation error

24. Final AI Summary

The AI in MOMO Quant should behave like a trading analyst, not like an uncontrolled trader.

It should help answer:

What market condition are we in?
Which strategy fits this condition?

How strong is the setup?
Should confidence be reduced?
Is the market abnormal?
Why was the trade taken?
Why was the trade rejected?
Which parameters are performing better?

The final execution decision still belongs to:

Strategy Engine
+
AI Confidence Engine
+
Risk Engine
+
Execution Engine

The most important rule:

AI advises. Risk controls. Execution obeys.